Indian Institute of Technology, Jodhpur
2025-26

Resilient State Machine Replication using Paxos and PBFT

Fundamentals of Distributed Systems

Assignment 1

Submitted By: Shifali Chandra
Roll Number: G25AI1040

1. Objective

The objective of this assignment is to design and implement a resilient distributed transaction ledger capable of handeling both crash faults and Byzantine faults.
The system integrates:

- Leader Election
- Process Coordination
- Paxos Consensus
- Practical Byzantine Fault Tolerance (PBFT)
- Digital Signatures
- Byzantine Fault Simulation
- Docker-based Deployment
- Chaos Testing

The implementation consists of a cluster of five distributed nodes that maintain a replicated transaction ledger while preserving consistency and fault tolerance.

2. System Architecture

Components

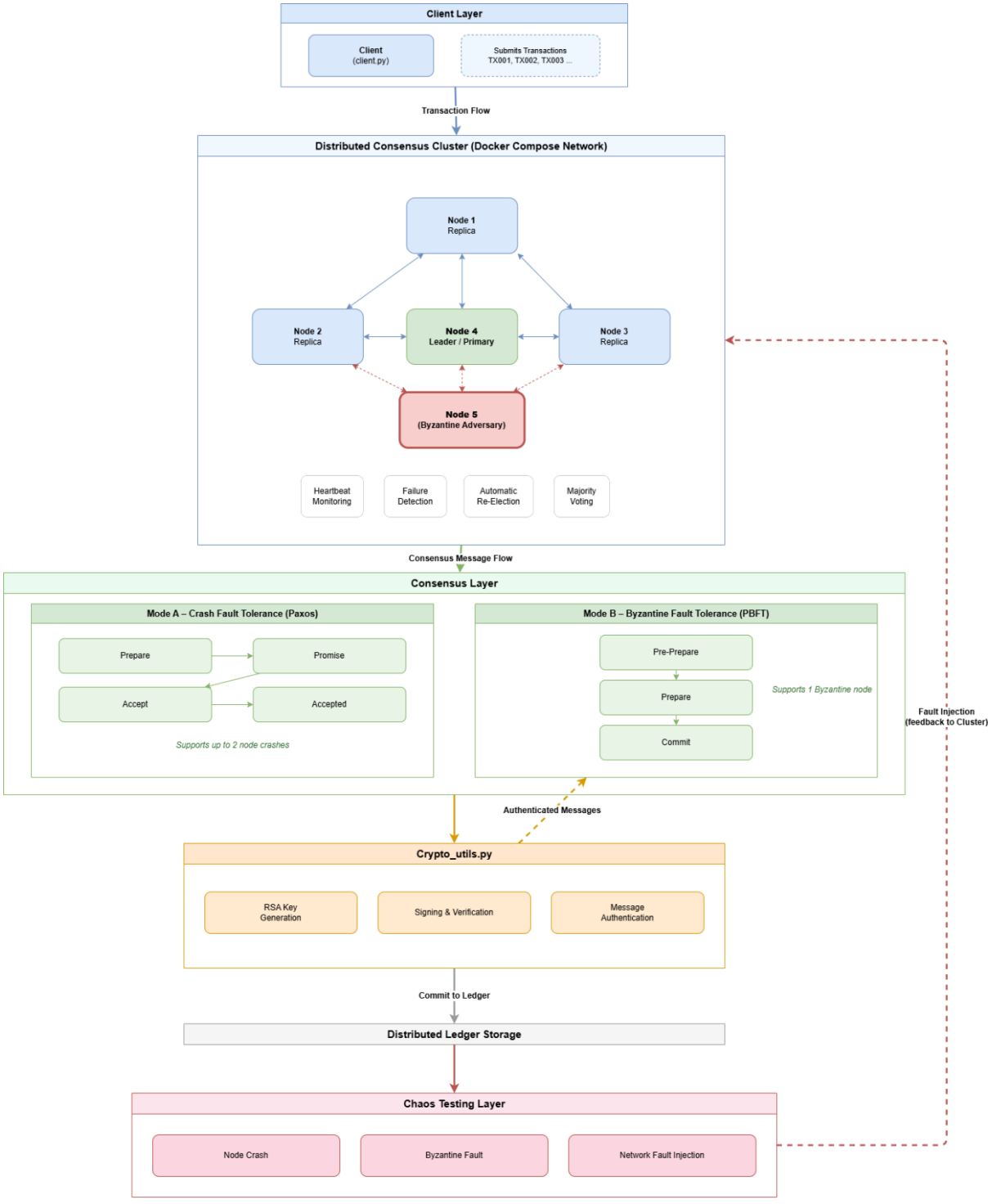
The system consists of:

1. Client Simulator
2. Five Distributed Nodes
3. Leader Election Module
4. Paxos Consensus Engine
5. PBFT Consensus Engine
6. Cryptographic Signature Module
7. Byzantine Adversary Node
8. Docker-Based Cluster Deployment
9. Chaos Testing Framework

Architecture Overview

The system follows a replicated state machine architecture consisting of five distributed nodes and one client. Client transactions are submitted to the cluster leader, which coordinates consensus among replicas. In Crash Fault Tolerance mode, the leader acts as the Paxos proposer and drives the Prepare, Promise, Accept, and Accepted phases. In Byzantine Fault Tolerance mode, the leader acts as the PBFT primary and coordinates the Pre-Prepare, Prepare, and Commit phases. Committed transactions are written to node-specific ledger files only after consensus is achieved. Docker Compose provides cluster orchestration, while chaos testing validates fault recovery under node failures and network disruptions.

Architecture Diagram



3. Process Coordination and Leader Election

Design

A heartbeat-based leader election mechanism is implemented.
Each node operates in one of three states:

- Follower
- Candidate
- Leader

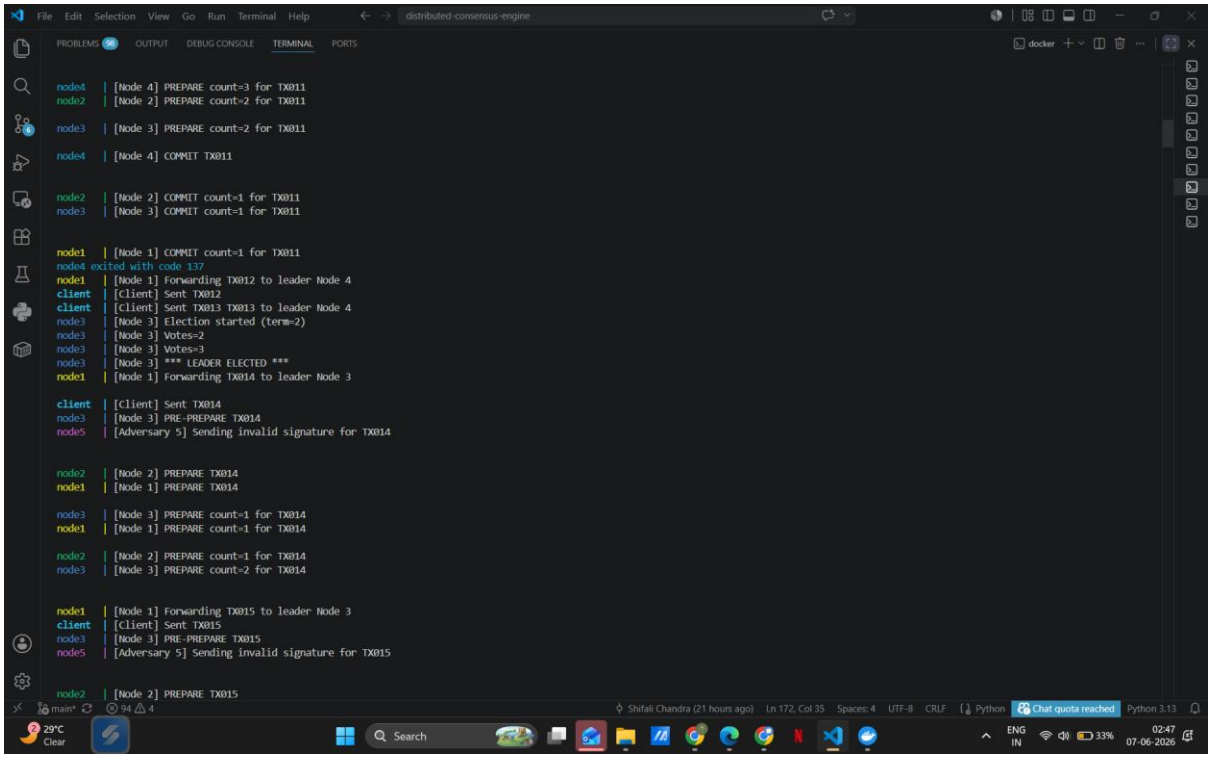
The leader periodically broadcasts heartbeat messages to all followers.
If a follower does not receive a heartbeat within the election timeout period:

1. It becomes a candidate.
2. Starts a new election term.
3. Requests votes from peers.
4. Becomes leader after receiving majority votes.

The elected leader acts as:

- Proposer in Paxos Mode
- Primary in PBFT Mode

Sample Output



Example:
Node 1 Election started (term=1)
Votes=2
Votes=3
*** LEADER ELECTED ***

Node State Transitions

Each node operates in one of three states:

- Follower → Candidate → Leader

A node begins as a follower. If heartbeat messages from the leader are not received within the election timeout period, the node transitions to the Candidate state and initiates an election. Upon receiving votes from a majority of nodes, the candidate transitions to the Leader state and begins broadcasting heartbeats. If a leader fails, another follower automatically transitions to candidate and the election process repeats.

4. Paxos Consensus Implementation

Overview

Crash fault tolerance is implemented using Basic Paxos.
Consensus proceeds through the following phases:

Phase 1: Prepare

Leader sends PREPARE request.

Phase 2: Promise

Acceptors reply with PROMISE messages.

Phase 3: Accept

Leader sends ACCEPT request after majority promises.

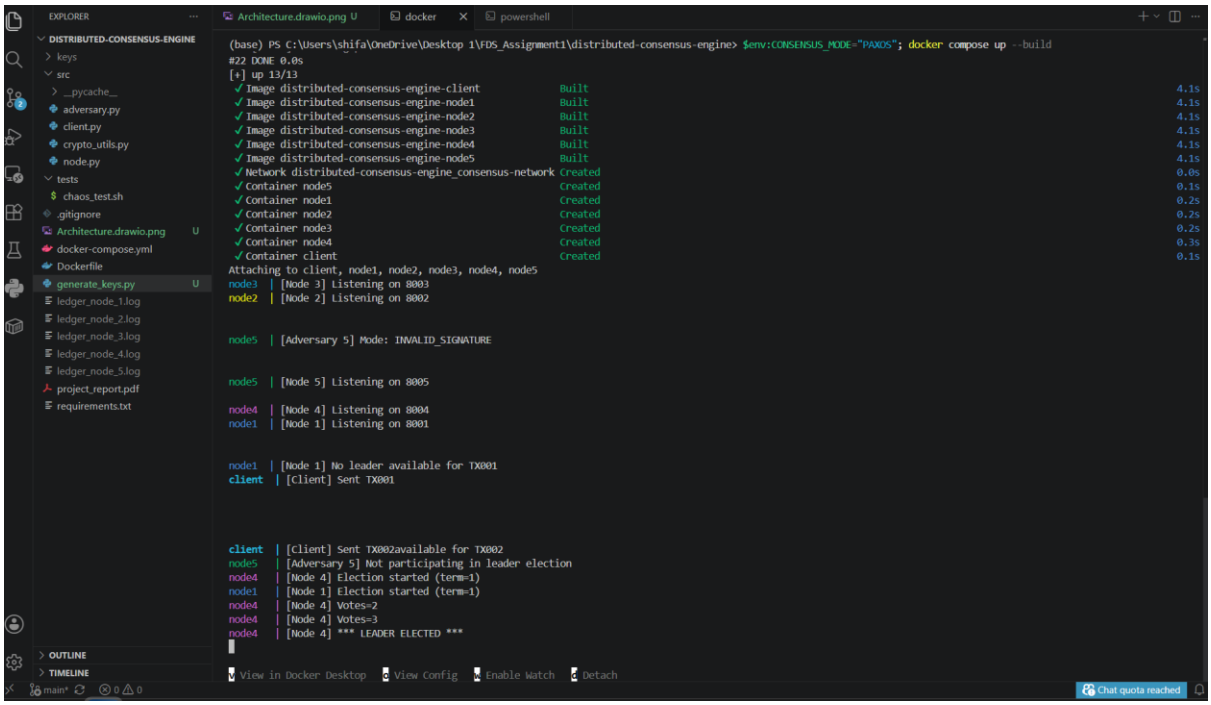
Phase 4: Accepted

Acceptors send ACCEPTED messages.

Commit

Transaction is committed after majority acceptance.

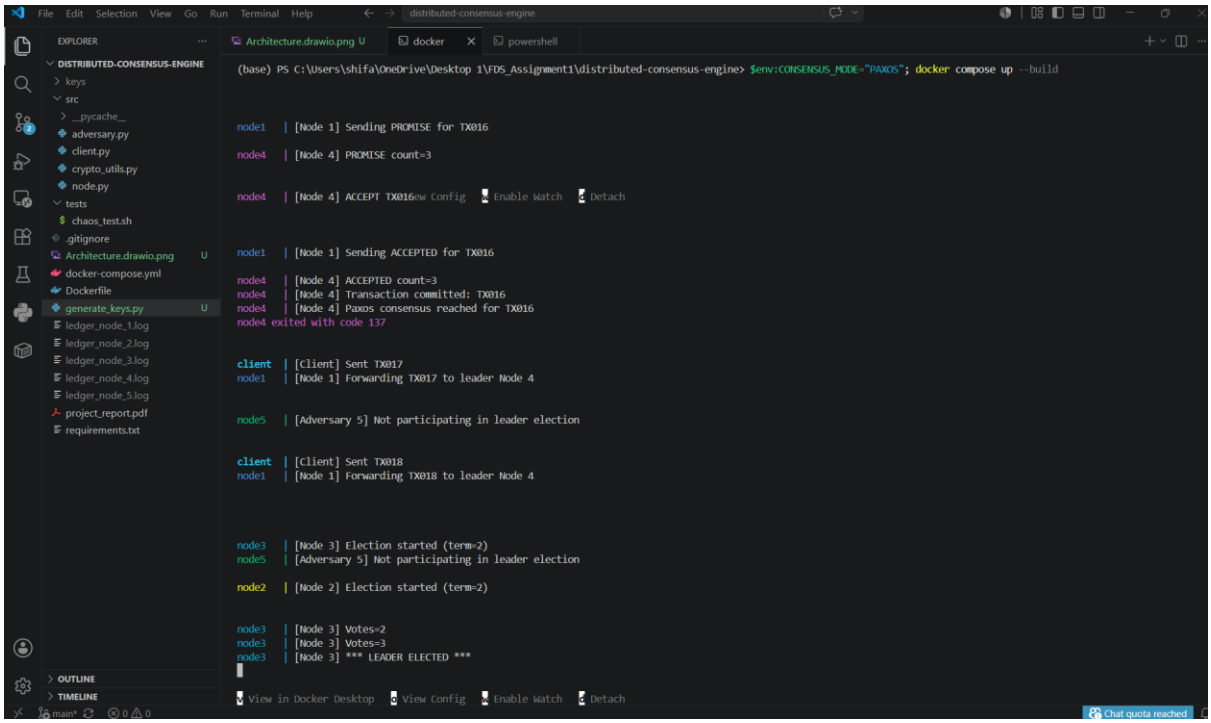
Sample Output



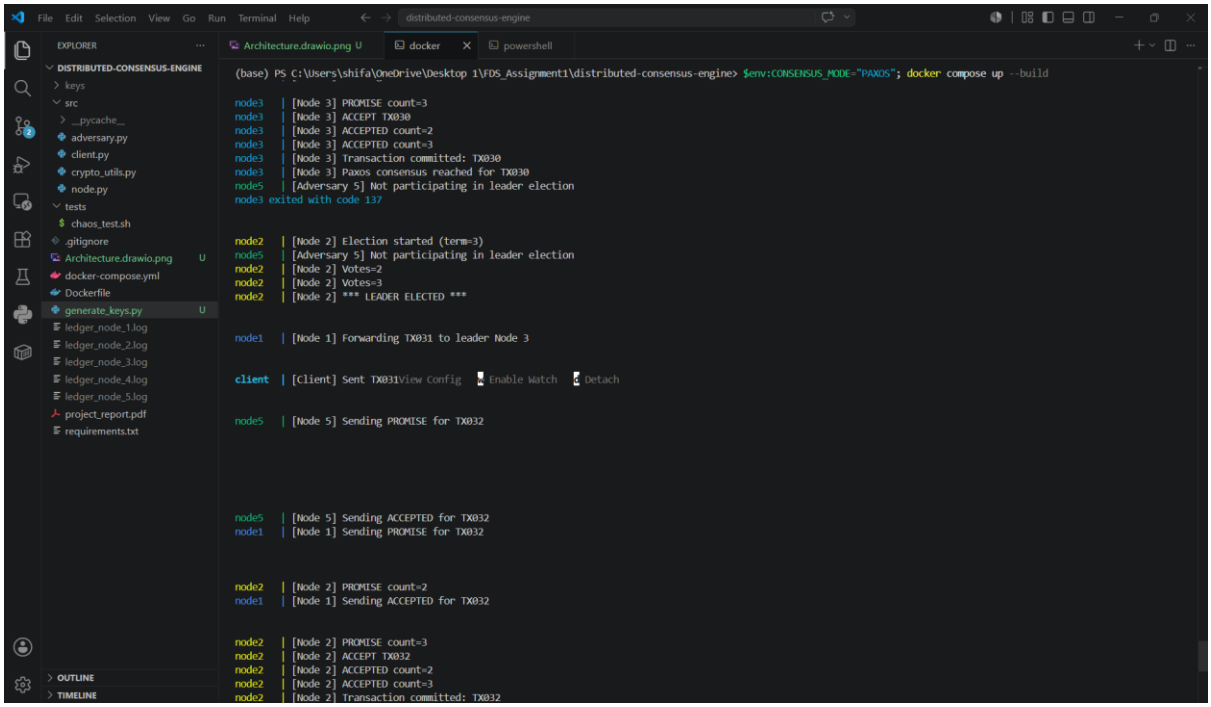
Fault Tolerance

The Paxos implementation tolerates crash failures while maintaining transaction consistency.

On stopping leader –> node4 -



On stopping new leader -> node 3 -



Thus, this shows two nodes have been stopped to simulate crash failures. The remaining majority continues processing transactions successfully.

5. PBFT Consensus Implementation

Overview

PBFT mode is used to tolerate Byzantine faults.

The protocol follows three phases:

Phase 1: Pre-Prepare

Primary proposes a transaction.

Phase 2: Prepare

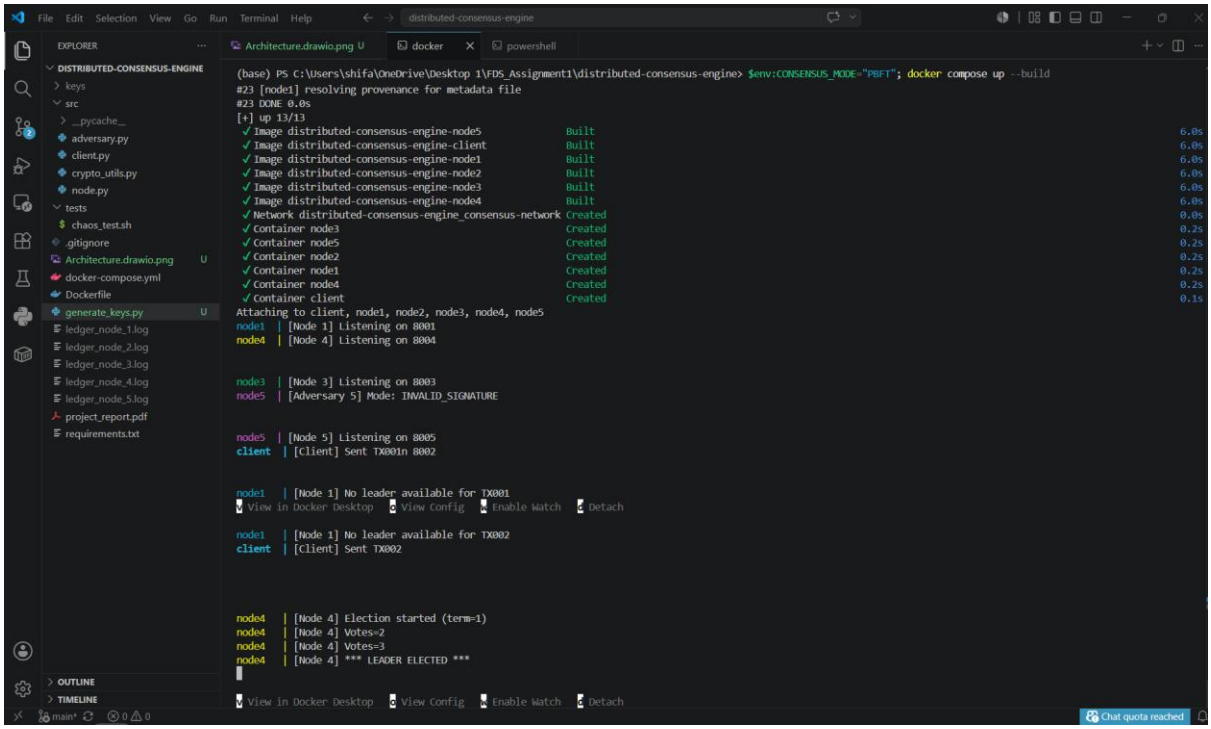
Replicas validate and broadcast prepare messages.

Phase 3: Commit

Nodes exchange commit messages.

Transaction is committed after receiving the required quorum.

Sample Output



Example:

PRE-PREPARE TX001

PREPARE count=1

PREPARE count=2

PREPARE count=3

COMMIT TX001

COMMIT count=1

COMMIT count=2

COMMIT count=3

Transaction committed: TX001

PBFT consensus reached for TX001

Byzantine Fault Tolerance

The system continues operation despite one malicious node in a five-node cluster.

6. Cryptographic Security

Digital Signatures

RSA public-private key pairs are generated for each node.
Each consensus message is:

1. Digitally signed by sender.
2. Verified by receiver.

Security Benefits

- Authentication
- Integrity
- Non-repudiation

Invalid signatures are rejected before participation in consensus.

Key Generation

Keys are generated using:
generate_key_pair(node_id)
Each node stores:

- node_X_private.pem
- node_X_public.pem

Key Distribution

Each node generates an RSA public-private key pair during initialization. Public keys are shared with all participating replicas and are used to verify signatures received from peers. Private keys never leave the originating node and are used only for signing outgoing consensus messages. This mechanism prevents message spoofing and ensures that PBFT replicas can verify message authenticity before participating in consensus.

7. Byzantine Adversary Simulation

Objective

To simulate malicious behavior in PBFT mode.
A dedicated adversary node is implemented.

Supported Attack Modes

Invalid Signature Attack

Node sends forged signatures.
Example:
[Adversary 5] Sending invalid signature for TX001

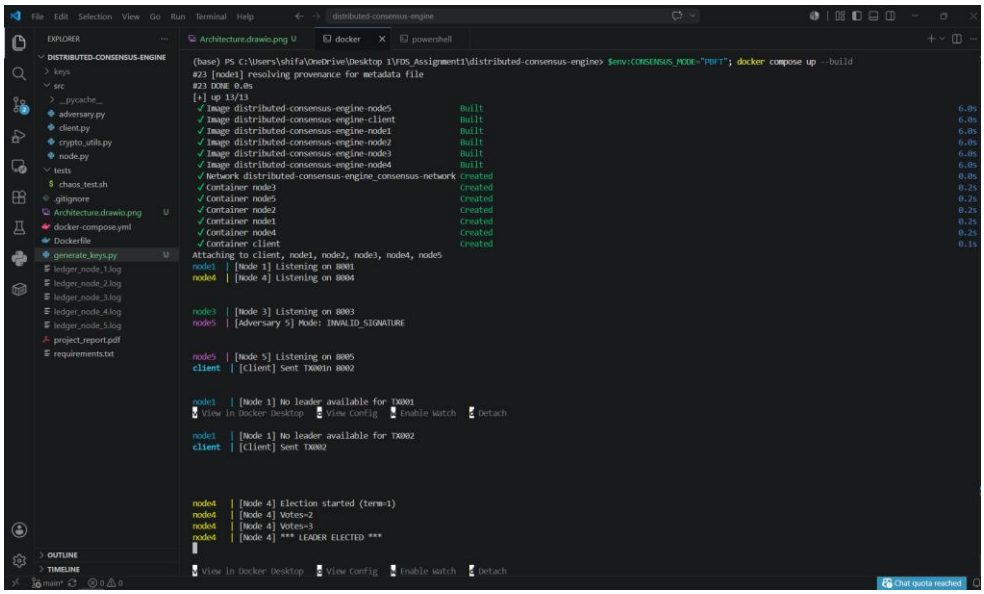
Drop Commit Attack

Node intentionally drops commit messages.

Equivocation Attack

Node sends conflicting transaction information to different peers.

Sample Output



The honest nodes continue processing transactions despite malicious behavior.

8. Docker-Based Deployment

Dockerfile

A Docker image is created for all nodes.

The image includes:

- Python Runtime
- Dependencies
- Consensus Engine
- Client Simulator

Docker Compose

Docker Compose is used to deploy:

- Node 1
- Node 2
- Node 3
- Node 4
- Node 5 (Byzantine Node)
- Client Container

Benefits

- Reproducibility
- Isolation
- Simplified Deployment
- Multi-node Simulation

9. Chaos Testing

Objective
To validate fault recovery mechanisms.
A chaos testing script is implemented.

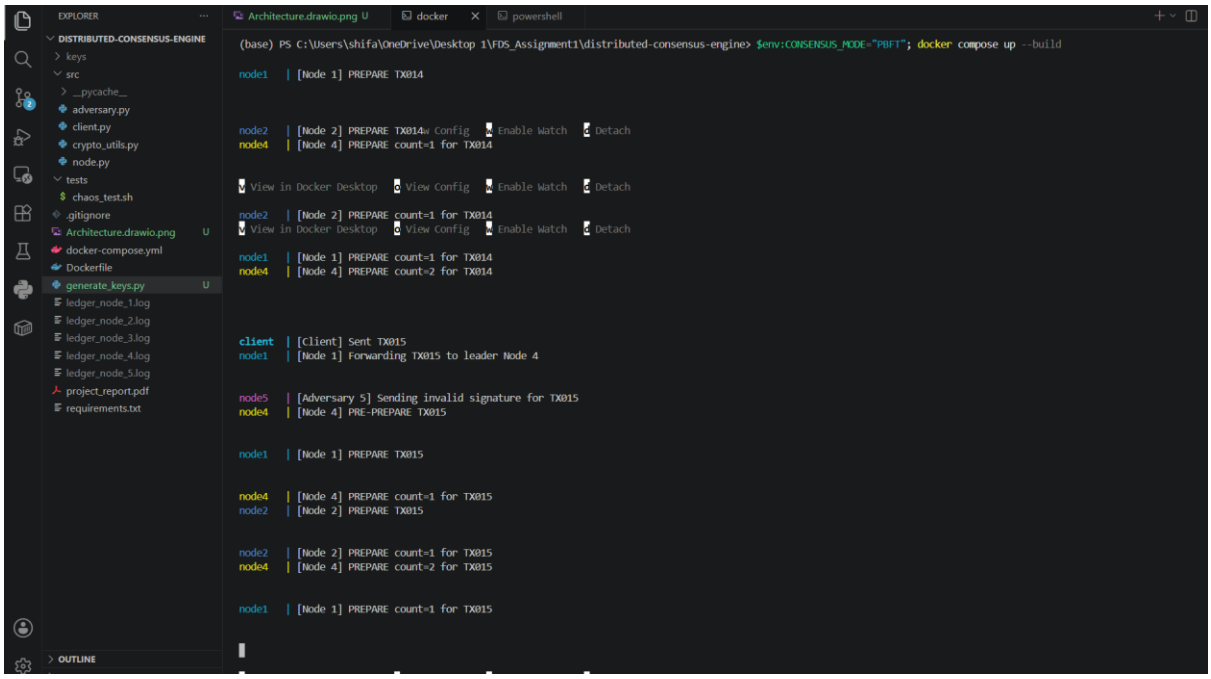
Scenario 1
Stop Leader Node
Expected Result:

- Followers detect failure.
- New election occurs.
- New leader elected.

Scenario 2
Restart Failed Node
Expected Result:

- Node rejoins cluster.
- Consensus continues.

Sample Output



Example: Stopping node3
Election started (term=3)
Votes=2
Votes=3
*** LEADER ELECTED ***
Consensus continues successfully.

Chaos Evaluation Result

During testing, the active leader node was intentionally terminated using Docker. The remaining replicas detected the failure through missed heartbeat messages and initiated a new election. A replacement leader was elected using majority voting, after which transaction processing resumed successfully. This demonstrates that the cluster can recover from node failures without manual intervention while preserving consensus and service availability.

10. Results

Leader Election

Successfully implemented and validated.

Paxos

Prepare, Promise, Accept, and Accepted phases executed successfully.

PBFT

Pre-Prepare, Prepare, and Commit phases executed successfully.

Byzantine Fault Tolerance

System remained operational despite malicious node behavior.

Docker Deployment

Successfully deployed and tested using Docker Compose.

Chaos Testing

Leader failures were recovered automatically through re-election.

11. Challenges Faced

- Preventing multiple simultaneous leaders.
- Synchronizing asynchronous consensus messages.
- Managing quorum calculations.
- Handling Byzantine behavior correctly.
- Docker networking configuration.
- Ensuring successful leader re-election after failures.

12. Conclusion

A resilient distributed transaction ledger was successfully implemented using Python, asyncio, Docker, Paxos, and PBFT.

The system demonstrates:

- Reliable leader election
- Crash fault tolerance through Paxos
- Byzantine fault tolerance through PBFT
- Secure communication using digital signatures
- Recovery from node failures
- Containerized deployment using Docker

The final implementation satisfies the requirements of the assignment and demonstrates practical distributed consensus under both crash-fault and Byzantine-fault environments.

13. GitHub Repository

Repository Link: https://github.com/Shifali-Chandra/distributed-consensus-engine

14. Video Demonstration

Video Link:
https://drive.google.com/file/d/1k3shgJbz4WeKPjIpdRL3mN3jwdylZgWw/view?usp=sharing